

```

// JARVIS on the iPhone: local language model + instant commands + local tools.
// Runs entirely in the phone (WebGPU via WebLLM, CPU fallback via wllama). The M
import { stockSymbol, stock } from './make.js';

export const WEBLLM_URL = 'https://cdn.jsdelivr.net/npm/@mlc-ai/web-llm@0.2.85/+e
export const WLLAMA_URL = 'https://cdn.jsdelivr.net/npm/@wllama/wllama@3.8.1/esm/
export const WLLAMA_WASM_URL = 'https://cdn.jsdelivr.net/npm/@wllama/wllama@3.8.1

export const MODELS = {
  fast: { label: 'Schnell · Qwen3 0.6B (~0,5 GB)', webllm: 'Qwen3-0.6B-q4f16_1-ML
  smart: { label: 'Klug · Qwen3 1.7B (~1,2 GB)', webllm: 'Qwen3-1.7B-q4f16_1-MLC'
  best: { label: 'Am klügsten · Qwen3 4B (~2,4 GB, nur iPhone 15 Pro+)', webllm:
};

// ----- engine
export class Brain {
  constructor() { this.engine = null; this.kind = null; this.model = null; this.b
  get ready() { return !!this.engine; }

  static hasWebGPU() { return typeof navigator !== 'undefined' && !!navigator.gpu

  async load(choice = 'smart', onProgress = () => {}) {
    const m = MODELS[choice] || MODELS.smart;
    let gpuError = null;
    if (Brain.hasWebGPU()) {
      try {
        onProgress(0.01, 'Prüfe Grafikchip ...');
        const adapter = await navigator.gpu.requestAdapter();
        if (!adapter) throw new Error('Kein GPU-Adapter');
        const f16 = adapter.features && adapter.features.has('shader-f16');
        onProgress(0.02, 'Lade KI-Engine (einmalig ca. 6 MB) ...');
        const webllm = await import(WEBLLM_URL);
        const ids = (webllm.prebuiltAppConfig?.model_list || []).map((x) => x.mod
        let want = f16 ? m.webllm : m.webllm.replace('q4f16_1', 'q4f32_1');
        if (!ids.includes(want)) want = ids.includes(m.webllm) ? m.webllm : (ids.
        onProgress(0.03, `Lade Modell ${want} ...`);
        this.engine = await webllm.CreateMLCEngine(want, { initProgressCallback:
        this.kind = 'webgpu'; this.model = want;
        return;
      } catch (e) {
        gpuError = e;
        console.warn('WebLLM fehlgeschlagen, nutze CPU', e);
        onProgress(0, 'GPU nicht nutzbar (' + String(e.message || e).slice(0, 80)

```

```

    }
  }
  try {
    onProgress(0.01, 'Lade KI-Engine (CPU) ...');
    const { Wllama } = await import(WLLAMA_URL);
    const w = new Wllama({ default: WLLAMA_WASM_URL });
    await w.loadModelFromHF({ repo: m.gguf.repo, file: m.gguf.file }, {
      n_ctx: 4096,
      progressCallback: ({ loaded, total }) => onProgress(total ? loaded / tota
    });
    this.engine = { wllama: w };
    this.kind = 'cpu'; this.model = m.gguf.file;
  } catch (e) {
    throw new Error((gpuError ? `GPU: ${gpuError.message} || gpuError`. ` : ''))
  }
}

/** Streams an answer; returns the full text. */
async chat(messages, { onDelta = () => {}, maxTokens = 320, signal } = {}) {
  if (!this.engine) throw new Error('Gehirn nicht geladen');
  this.busy = true;
  let raw = '';
  try {
    if (this.kind === 'webgpu') {
      const it = await this.engine.chat.completions.create({ messages, stream:
      for await (const ch of it) {
        if (signal?.aborted) { try { this.engine.interruptGenerate(); } catch (
        const d = ch.choices?.[0]?.delta?.content || '';
        if (d) { raw += d; onDelta(visible(raw)); }
      }
    } else {
      const it = await this.engine.wllama.createChatCompletion({ messages, stre
      for await (const ch of it) {
        if (signal?.aborted) break;
        const d = ch.choices?.[0]?.delta?.content ?? ch.choices?.[0]?.text ?? '
        if (d) { raw += d; onDelta(visible(raw)); }
      }
    }
  } finally { this.busy = false; }
  return visible(raw);
}
}

/** Removes <think> blocks (also unfinished ones) from model output. */
export function visible(raw) {
  let s = raw.replace(/<think>[\s\S]*?<\/think>/g, '');

```

```

    const open = s.indexOf('<think>');
    if (open >= 0) s = s.slice(0, open);
    return s.replace(/^\s+/, '');
}

// ----- memory (local storage) -----
export const Memory = {
  key: 'jarvisMemory',
  all() { try { return JSON.parse(localStorage.getItem(this.key) || '[]'); } catch {
  save(items) { try { localStorage.setItem(this.key, JSON.stringify(items.slice(-
  add(text, category = 'notiz') {
    const items = this.all();
    const n = norm(text);
    if (items.some((i) => norm(i.text) === n)) return false;
    items.push({ text, category, created: new Date().toISOString() });
    this.save(items); return true;
  },
  pref(k, v) {
    const p = JSON.parse(localStorage.getItem('jarvisPrefs') || '{}');
    if (v === undefined) return p[k];
    p[k] = v; localStorage.setItem('jarvisPrefs', JSON.stringify(p)); return v;
  },
  promptBlock() {
    const p = JSON.parse(localStorage.getItem('jarvisPrefs') || '{}');
    const lines = Object.entries(p).map(([k, v]) => `- ${k}: ${v}`).concat(this.all());
    return lines.length ? lines.join('\n') : '(noch keine)';
  },
};

const norm = (s) => String(s).toLowerCase().normalize('NFD').replace(/[^\w]/g, '');

export function systemPrompt(title = 'Master') {
  const now = new Date().toLocaleString('de-DE', { weekday: 'long', day: 'numeric' });
  return `Du bist JARVIS, der persönliche KI-Assistent deines Nutzers, und läufst
  Deine Antworten werden vorgelesen: 1-3 kurze Sätze, kein Markdown, keine Listen,
  Wenn du etwas Aktuelles nicht weißt (Kurse, Nachrichten), sag das ehrlich statt z
  Du kannst auf dem iPhone selbst Präsentationen, PDFs, Word- und Excel-Dateien, We
  Jetzt: ${now}.
  Was du über den Nutzer weißt:
  ${Memory.promptBlock()} /no_think`
}

// ----- instant commands -----
const CRYPTO = { bitcoin: 'BTC', btc: 'BTC', ethereum: 'ETH', eth: 'ETH', solana:
// Only things the phone can't do itself (they control the Mac).
const MAC_ONLY = ['lautstärke', 'ordner', 'löscher', 'verschieben', 'programm', 'terminal']
const GEN = [

```

```

['video', /(tiktok|reel|short|video|clip)/],
['pptx', /(präsentation|praesentation|folien|powerpoint|pptx|slides|vortrag)/],
['html', /(website|webseite|homepage|landingpage|landing page|internetseite|web
['xlsx', /(tabelle|excel|xlsx|spreadsheet|liste als tabelle)/],
['docx', /( word|docx|word-dokument|word dokument)/],
['pdf', /(pdf|dokument|bericht|aufsatz|brief|zusammenfassung|referat|handout|ko
];
const CREATE = /(erstell|mach |mache |bau |baue |schreib|generier|produzier|entwi

function cap(s, re) { const m = s.match(re); return m ? m[1].trim() : null; }
const title = (s) => s.split(' ').map((w) => w.charAt(0).toUpperCase() + w.slice(

/** Extracts the subject of a "make me a ... about X" request. */
export function topicOf(text) {
  const t = text.replace(/[\?!.]+\$/ , '').replace(/\s+(als|im)\s+(pdf|word|excel|po
  const m = t.match(/(?:^\s)(?:über|zum thema|zu dem thema|thema|zur|zum|zu|für|
  if (m) return m[1].replace(/^(den|die|das|dem|der|eine?n?)\s+(?=\S+\s)/i, '').t
  return t.replace(/^(jarvis,?\s*)?(bitte\s+)?/i, '')
    .replace(/\b(erstell\w*|mach\w*|bau\w*|schreib\w*|generier\w*|produzier\w*|en
    .replace(/\s+/g, ' ').trim() || 'Allgemein';
}

/** Returns {local:'tool', args} | {gen:kind, topic} | {mac:true} | {reply} | nul
export function route(text, { macAvailable = false } = {}) {
  const s = ' ' + text.toLowerCase().replace(/[^a-zäöüß0-9.&\- ]/g, ' ').replace(
  const has = (...w) => w.some((x) => s.includes(x));
  if (has('wie spät', 'uhrzeit', 'wieviel uhr', 'wie viel uhr')) return { reply:
  if (has('welcher tag', 'welches datum', 'datum heute')) return { reply: `Heute
  const fact = cap(s, /^ ?(?:jarvis )?(?:bitte )?(?:merk|merke|speicher|notier) (
  if (fact) return { local: 'remember', args: { fact: text.replace(/^.*(?:dass|d
  const place = cap(s, /(?:ich wohne in|mein wohnort ist|ich lebe in) ([a-zäöüß
  if (place) return { local: 'home', args: { place: title(place) } };
  // posting / sharing the last file (TikTok via the iOS share sheet)
  if (has('poste', 'posten', 'hochladen', 'lade es hoch', 'teile', 'teilen', 'ver
  const site = cap(s, /^ ?(?:jarvis )?(?:öffne|starte|geh auf|gehe auf) (youtube|
  if (site) return { local: 'openSite', args: { site } };
  if (CREATE.test(s)) {
    const g = GEN.find(([, re]) => re.test(s));
    if (g) return { gen: g[0], topic: topicOf(text) };
  }
  if (has('wetter', 'temperatur', ' regnet', 'wie warm', 'wie kalt')) {
    const loc = cap(s, /(?:in|für|bei) ([a-zäöüß .-]+?)(?: morgen| heute| übermo
    return { local: 'weather', args: { location: loc ? title(loc.replace(/^(der|d
  }
  const wantsAnalysis = has('tipp', 'einschätzung', 'analyse', 'analysier', 'sign
  if (has('krypto', 'crypto', 'coins')) {

```

```

    const one = Object.keys(CRYPTO).find((k) => s.includes(' ' + k + ' '));
    if (!one) return { local: 'cryptoScan', args: {} };
  }
  const coin = Object.keys(CRYPTO).sort((a, b) => b.length - a.length).find((k) =>
  if (coin) return { local: wantsAnalysis ? 'cryptoAnalysis' : 'cryptoQuote', arg
  if (has(' aktie', 'dax', 'nasdaq', 'dow jones', 's&p', 'börsen', 'kurs', 'goldp
    const st = stockSymbol(text);
    if (st) return { local: wantsAnalysis ? 'stockAnalysis' : 'stockQuote', args:
  }
  if (has('trading tipp', 'trading-tipp', 'tipps', 'was soll ich traden', 'was so
  if (has('nachrichten', 'news', 'was passiert', 'was ist los', 'schlagzeilen',
    const r = cap(s, / (?:(in|im|aus|über|zu) (?:(der |dem |den |die )?([a-zäöüß .-
    return { local: 'news', args: { region: r && r !== 'welt' ? title(r) : null }
  }
  const song = cap(s, /^ ?(?:jarvis )?(?:spiel|spiele|play|hör|höre) (?:(mal |bitt
  if (song && !has(...MAC_ONLY)) return { local: 'music', args: { query: song } }
  const q = cap(s, /^ ?(?:jarvis )?(?:such|suche|google|googel|recherchier\w*)(?:
  if (q && !has('datei', 'ordner')) return { local: 'webSearch', args: { query: q
  if (has(...MAC_ONLY)) return macAvailable ? { mac: true } : { reply: 'Das betri
  const any = cap(s, /^ ?(?:jarvis )?(?:öffne|starte) (.*?) ?$/);
  if (any) return { local: 'webSearch', args: { query: any } };
  return null;
}

// ----- local tools
async function getJSON(url) {
  const r = await fetch(url);
  if (!r.ok) throw new Error(`HTTP ${r.status}`);
  return r.json();
}

const WMO = { 0: 'klar', 1: 'überwiegend klar', 2: 'teils bewölkt', 3: 'bedeckt',

export async function weather(location) {
  const loc = location || Memory.pref('wohntort') || 'Berlin';
  const g = await getJSON(`https://geocoding-api.open-meteo.com/v1/search?count=1
  const r = g.results && g.results[0];
  if (!r) throw new Error(`Ort ${loc} nicht gefunden`);
  const w = await getJSON(`https://api.open-meteo.com/v1/forecast?latitude=${r.la
  const c = w.current, d = w.daily;
  const data = { place: [r.name, r.country].filter(Boolean).join(', '), current:
  const say = `In ${r.name} sind es ${Math.round(c.temperature_2m)} Grad, ${WMO[c
  return { say, view: 'weather', data };
}

async function klines(sym, interval = '1d', limit = 260) {

```

```

try {
  const k = await getJSON(`https://api.binance.com/api/v3/klines?symbol=${sym}U
  return k.map((x) => ({ t: x[0], o: +x[1], h: +x[2], l: +x[3], c: +x[4], v: +x
} catch (e) {
  // fallback: CoinGecko daily closes (needs the coin id)
  const ids = { BTC: 'bitcoin', ETH: 'ethereum', SOL: 'solana', XRP: 'ripple',
  const j = await getJSON(`https://api.coingecko.com/api/v3/coins/${ids[sym]}/m
  return j.prices.map(([t, c], i, a) => ({ t, o: i ? a[i - 1][1] : c, h: c, l:
}
}

const sma = (a, n) => (a.length >= n ? a.slice(-n).reduce((x, y) => x + y, 0) / n
function rsi(c, n = 14) {
  if (c.length <= n) return null;
  let g = 0, l = 0;
  for (let i = 1; i <= n; i++) { const d = c[i] - c[i - 1]; if (d > 0) g += d; el
  g /= n; l /= n;
  for (let i = n + 1; i < c.length; i++) { const d = c[i] - c[i - 1]; g = (g * (n
  return l === 0 ? 100 : 100 - 100 / (1 + g / l);
}

/** Same rules as the Mac (Trading.swift): trend, momentum, RSI → signal with ent
export function analyze(k) {
  const c = k.map((x) => x.c), price = c[c.length - 1];
  const s20 = sma(c, 20), s50 = sma(c, 50), s200 = sma(c, 200), r = rsi(c);
  const trs = k.slice(1).map((x, i) => Math.max(x.h - x.l, Math.abs(x.h - k[i].c)
  const atr = trs.length >= 14 ? trs.slice(-14).reduce((a, b) => a + b, 0) / 14 :
  const mom = c.length > 21 ? (price / c[c.length - 21] - 1) * 100 : null;
  const recent = k.slice(-20), support = Math.min(...recent.map((x) => x.l)), res
  let score = 0; const reasons = [];
  if (s50) { if (price > s50) { score++; reasons.push('Kurs über dem 50-Tage-Schn
  if (s20 && s50) { if (s20 > s50) { score++; reasons.push('kurzfristiger Trend a
  if (s50 && s200) { if (s50 > s200) { score++; reasons.push('langfristiger Aufwä
  if (mom !== null) { if (mom > 2) { score++; reasons.push(`Momentum plus ${mom.to
  if (r !== null) { if (r < 30) { score++; reasons.push(`RSI ${Math.round(r)}, übe
  const signal = score >= 2 ? 'bullisch' : score <= -2 ? 'bärisch' : 'neutral';
  let entry = price, stop, target;
  if (signal === 'bullisch') { if (s20 && price > s20 * 1.04) entry = s20 * 1.01;
  else if (signal === 'bärisch') { stop = Math.min(Math.max(resistance, entry + a
  else { stop = support; target = resistance; }
  return { signal, score, entry, stop, target, support, resistance, rr: Math.abs(
}

const fmt = (v) => (v >= 1000 ? Math.round(v).toLocaleString('de-DE') : v >= 1 ?

export async function cryptoQuote(coin, withAnalysis = false) {

```

```

const k = await klines(coin);
const last = k[k.length - 1], prev = k[k.length - 2] || last;
const changePct = (last.c / prev.c - 1) * 100;
const an = analyze(k);
const data = { symbol: `${coin}-USD`, name: coin, currency: 'USD', price: last.c };
let say = `${coin} steht bei ${fmt(last.c)} Dollar, ${changePct >= 0 ? 'plus' : 'minus'} ${Math.abs(changePct).toFixed(1)}%`;
if (withAnalysis) {
  say = `${coin}: Signal ${an.signal}. ${an.reasons.slice(0, 3).join(', ')} `;
}
return { say, view: 'markets', data };
}

export async function cryptoScan() {
  const coins = ['BTC', 'ETH', 'SOL', 'XRP', 'DOGE', 'ADA', 'BNB'];
  const res = (await Promise.allSettled(coins.map(async (c) => { const k = await klines(c);
  if (!res.length) throw new Error('Keine Kursdaten erreichbar');
  res.sort((a, b) => b.an.score - a.an.score);
  const best = res.filter((x) => x.an.signal === 'bullisch').slice(0, 2);
  const top = res[0], last = top.k[top.k.length - 1], prev = top.k[top.k.length - 2];
  const say = (best.length ? 'Stärkste Kaufsignale: ' + best.map((x) => `${x.c} m`
  + (res[res.length - 1].an.signal === 'bärisch' ? ` Am schwächsten: ${res[res.length - 1].an.score} ` : ''));
  const data = { symbol: `${top.c}-USD`, name: top.c, currency: 'USD', price: last.c };
  watch: res.map((x) => { const l = x.k[x.k.length - 1], p = x.k[x.k.length - 2];
  return { say, view: 'markets', data };
}

export async function news(region) {
  const q = region ? `https://www.tagesschau.de/api2u/search/?pageSize=12&resultP` : `https://www.tagesschau.de/api2u/search/?pageSize=12&resultP`;
  const j = await getJSON(q);
  const list = (j.searchResults || j.news || []).filter((n) => n.title).slice(0, 10);
  const title = [n.topline, n.title].filter(Boolean).join(': ');
  const url = n.shareURL || n.teaserImage?.imageVariants?.['16x9-512'] || n.teaserImage?.imageVariants?.['16x9-512'];
  if (!list.length) throw new Error('Keine Meldungen gefunden');
  const say = `Die wichtigsten Meldungen${region ? ' zu ' + region : ''}: ` + list.map(n => `${n.title} ${n.teaserImage?.imageVariants?.['16x9-512'] || n.teaserImage?.imageVariants?.['16x9-512']}`);
  return { say, view: 'globe', data: { country: region || 'Welt', news: list, source: 'tagesschau.de' } };
}

export async function runLocal(tool, args) {
  switch (tool) {
    case 'remember': return { say: Memory.add(args.fact) ? 'Notiert.' : 'Das weiß ich schon.' };
    case 'home': Memory.pref('wohntort', args.place); return { say: 'Gespeichert' };
    case 'weather': return weather(args.location);
    case 'cryptoQuote': return cryptoQuote(args.coin, false);
    case 'cryptoAnalysis': return cryptoQuote(args.coin, true);
    case 'cryptoScan': return cryptoScan();
  }
}

```

```

    case 'news': return news(args.region);
    case 'stockQuote': return stock(args.symbol, args.name, false);
    case 'stockAnalysis': return stock(args.symbol, args.name, true);
    case 'music': window.open(`https://open.spotify.com/search/${encodeURIComponent
    case 'webSearch': window.open(`https://www.google.com/search?q=${encodeURIComponent
    case 'openSite': {
        const urls = { youtube: 'https://www.youtube.com', google: 'https://www.goo
        window.open(urls[args.site] || 'https://www.google.de', '_blank');
        return { say: 'Bitte sehr.' };
    }
    default: throw new Error('Unbekanntes Werkzeug');
}
}

/** Splits streamed text into speakable sentences. */
export class Sentences {
    constructor() { this.done = 0; }
    take(text, final = false) {
        const rest = text.slice(this.done);
        const out = [];
        const re = final ? /\s\S]*?[.!?...]+(?:\s|$)/g : /\s\S]*?[.!?...]+(?:\s)/g;
        let m, last = 0;
        while ((m = re.exec(rest))) { if (!m[0]) { re.lastIndex++; continue; } if (m[
        this.done += last;
        if (final) { const tail = text.slice(this.done).trim(); if (tail) out.push(ta
        return out;
    }
}

```